PROJECT #2

2.1. INTRODUCTION

This project presents the design and implementation of a 4-bit binary calculator in Verilog, capable of performing arithmetic operations—addition, subtraction, multiplication, and division—on 4-bit numbers. The calculator supports inputs ranging from 0 to 15 (treated as unsigned values), with a special consideration for signed interpretation in subtraction, where negative results are indicated via a dedicated flag. The design integrates a datapath with two 4-bit registers, and an Arithmetic Logic Unit (ALU), to show results up to 255 (8-bit unsigned). Key features include handling division by zero and providing status signals such as done, div_by_zero, and negative.

The objective is to create a functional calculator that processes 4-bit inputs and displays results in a human-readable format, suitable for educational purposes or as a building block in larger digital systems.

2.2. OBJECTIVE

• **Develop Verilog Coding Skills**: Create a datapath using structural Verilog for components like the decoder and registers, and behavioral Verilog for ALU operations such as multiplication and division.
• **Simulation**: Use ModelSim to simulate the datapath, testing all arithmetic operations and analyzing waveform outputs.
• **Report Preparation**: Document your code, simulation results, and implementation process in a comprehensive report.

2.3. MODULE DESCRIPTIONS

The calculator is composed of several modular components, each performing a specific function. Below is a detailed description of each module.

2.3.1. Decoder (decoder_1to2)

- **Purpose**: Decodes a 1-bit write address to generate a 2-bit select signal for loading data into one of two registers (A or B).
- **Inputs**:
  - write_addr (1-bit): Determines which register to select (0 for A, 1 for B).
- **Outputs**:
  - sel (2-bit): Select lines (2'b01 for A, 2'b10 for B).
- **Operation**: Uses a ternary operator to assign sel based on write_addr.

2.3.2. Multiplexers

- **mux2to1**:
  - **Purpose**: A 2-to-1 multiplexer for 4-bit data selection.
  - **Inputs**:
    - sel (1-bit): Select signal.
    - in0, in1 (4-bit each): Input data options.

- **Output**:
    - out (4-bit): Selected data (in0 if sel=0, in1 if sel=1).
- **mux2to1_1bit**:
    - **Purpose**: A 2-to-1 multiplexer for 1-bit data selection.
    - **Inputs**:
        - sel (1-bit): Select signal.
        - in0, in1 (1-bit each): Input options.
    - **Output**:
        - out (1-bit): Selected bit.

2.3.3. Registers (dff4)

- **Purpose**: A 4-bit D flip-flop register with clock and active-low reset.
- **Inputs**:
    - clk: Clock signal.
    - reset: Active-low reset (clears output to 4'b0 when low).
    - data_in (4-bit): Input data to store.
- **Output**:
    - data_out (4-bit): Stored data, updated on the positive clock edge.

2.3.4. Full Adder (full_adder_1bit)

- **Purpose**: Performs 1-bit addition with carry-in and carry-out.
- **Inputs**:
    - A, B (1-bit each): Operands.
    - Cin: Carry-in.
- **Outputs**:
    - Sum: Result of A + B + Cin.
    - Cout: Carry-out.
- **Operation**: Uses XOR and AND gates to compute sum and carry.

2.3.5. Ripple Carry Adder (ripple_carry_adder_4bit)

- **Purpose**: A 4-bit adder constructed from four 1-bit full adders.
- **Inputs**:
    - A, B (4-bit each): Operands.
    - Cin: Initial carry-in.
- **Outputs**:
    - Sum (4-bit): Sum of A + B + Cin.
    - Cout: Carry-out from the most significant bit.

2.3.6. Adder/Subtractor (adder_subtractor)

- **Purpose**: Performs 4-bit addition or subtraction, outputting an 8-bit result.
- **Inputs**:
    - A, B (4-bit each): Operands.
    - OP: Operation select (0 for addition, 1 for subtraction).

- **Output**:
  - S (8-bit): Result formatted as {3'b000, Cout, Sum}.
- **Operation**:
  - Addition (OP=0): Computes A + B, with Cout as the carry-out.
  - Subtraction (OP=1): Computes A - B using two's complement (A + ~B + 1). If A < B (detected via Cout=0), it calculates the magnitude |A - B|.
  - **Hint:** The OP signal is generated from the LSB of compute_op.

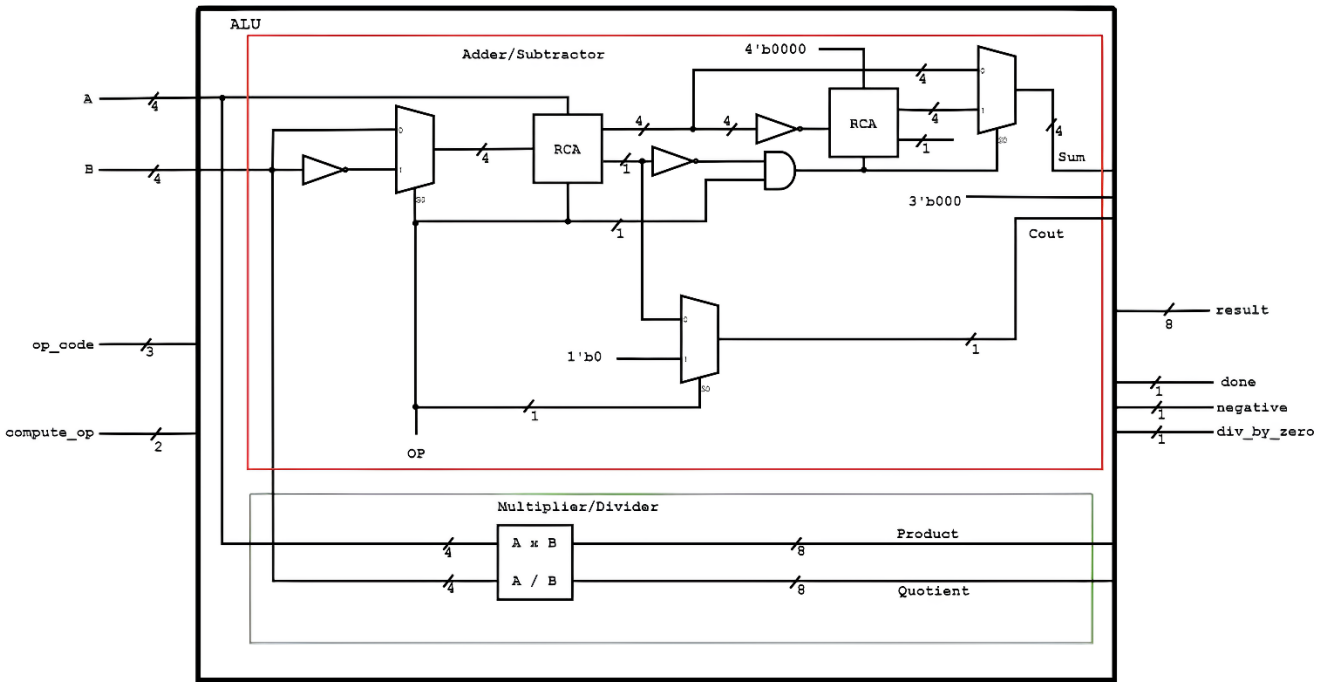

Figure 1: Arithmetic Logic Unit (ALU)

2.3.7. Arithmetic Logic Unit (ALU)

- **Purpose**: Executes arithmetic operations and manages data display.
- **Inputs**:
  - clk, reset: Clock and active-low reset.
  - A, B (4-bit each): Operands from registers.
  - op_code (3-bit): Command (e.g., LOAD_A, COMPUTE, DISPLAY_RES).
  - compute_op (2-bit): Operation for COMPUTE (00=ADD, 01=SUB, 10=MUL, 11=DIV).
- **Outputs**:
  - result (8-bit): Result to display.
  - done: Indicates completion of display operations.
  - div_by_zero: Flags division by zero.
  - negative: Indicates a negative result for subtraction (A < B).

- **Operations**:
  - **NOOP (000):** No operation/ Idle state
  - **LOAD_A (001)**: Loads A into reg_A.
  - **LOAD_B (011)**: Loads B into reg_B.
  - **COMPUTE (101)**:
    - ADD (00): Uses adder_subtractor.
    - SUB (01): Uses adder_subtractor, sets negative on display if A < B.
    - MUL (10): Unsigned multiplication (reg_A * reg_B).
    - DIV (11): Unsigned division ({reg_A % reg_B, reg_A / reg_B}), flags div_by_zero if reg_B=0.
  - **DISPLAY_A (010)**, **DISPLAY_B (100)**: Displays either A or B value
  - **DISPLAY_RES (110)**: Outputs final result values with done=1.
  - **RESET (111)**: Clears registers.

Table 1: ALU op_codes and instructions

op_code	Instruction
000	Noop
001	Load A
010	Display A
011	Load B
100	Display B
101	Compute
110	Display result
111	Noop

Table 2: Computation operation codes

compute_op	Operation
00	Add
01	Subtract
10	Multiply
11	Divide

2.3.8. Datapath (datapath)

- **Purpose**: Integrates registers, multiplexers, and the ALU to manage data flow.
- **Inputs**:
  - clk, data_in (4-bit), wrt_addr, op_code, compute_op.
- **Outputs**:
  - result, done, div_by_zero, negative.
- **Structure**: Uses a decoder and multiplexers to load data into registers reg0 (A) or reg1 (B), feeding them into the ALU.

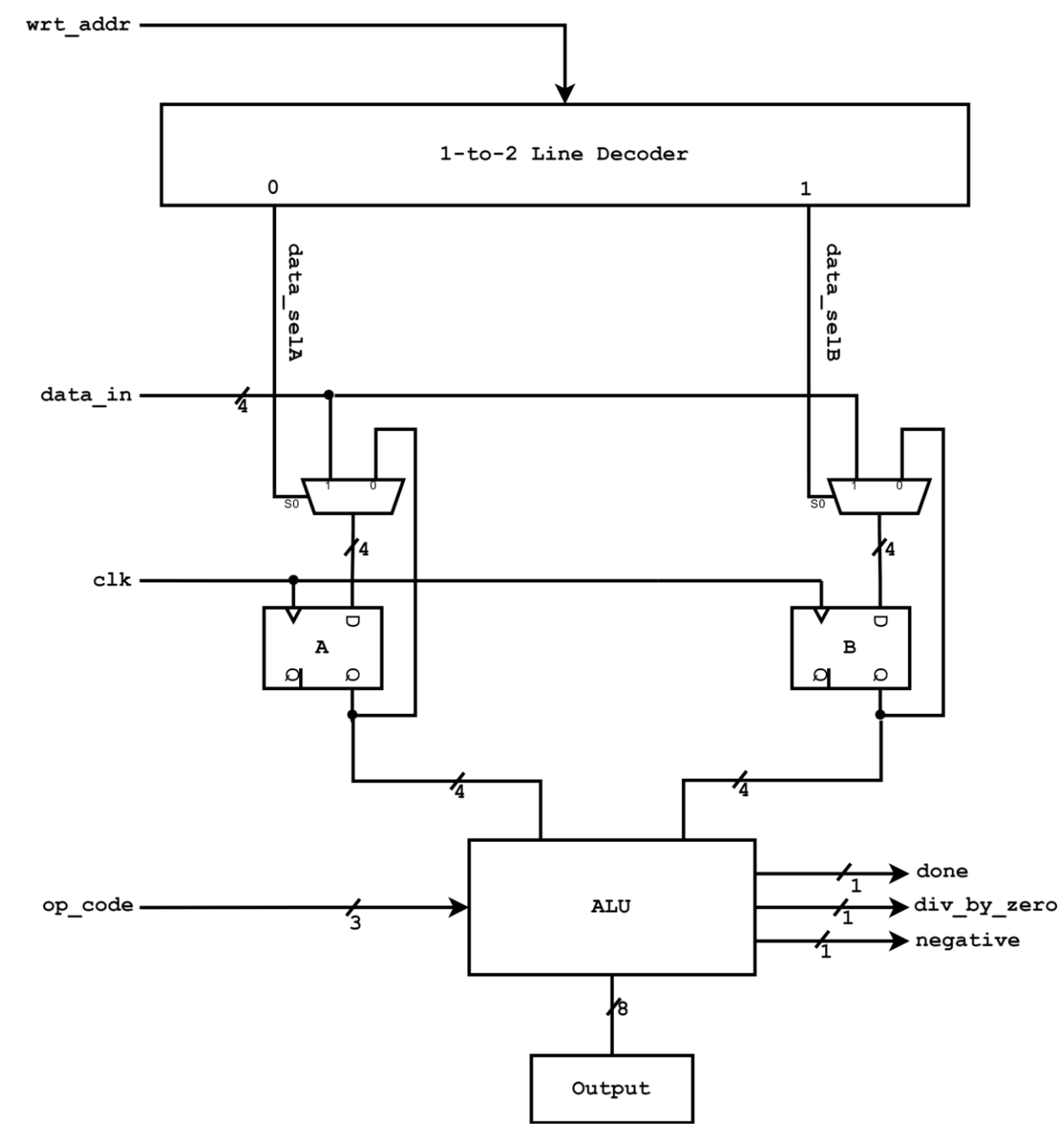

Figure 2: Datapath schematic

2.4. DATAPATH DESIGN AND SIMULATION

The ALU uses a 3-bit op_code input and a 2-bit internal compute_op wire to select the operation only when the compute operation is selected (Table 1 and 2 respectively). Addition and subtraction are to be implemented structurally using the provided adder/subtractor schematic (Figure 1), while multiplication and division will be designed using behavioral modeling (using numerical operators).

Create a new folder named project2 (do not include spaces). Open Modelsim and create a new project in this folder and name it project2.

Design the submodules (decoder, muxes, flip-flops) using any Verilog modeling style of your choice. It is advisable to write a test bench for every module you design to ensure proper functionality before up-scaling.

2.4.1. 4-bit Adder/Subtractor

A full Adder has three inputs, A, B, CIN (Carry-IN), and two outputs, SUM and COUT (Carry-OUT). The output represents the result from computing binary addition of the three inputs. Derive a 1-bit Full adder truth-table and write down the logic expressions for SUM and COUT.

Write a **behavioral** Verilog code to implement the full adder you have designed.

Use the symbol in Figure 3(a) for 1-bit Full-Adder and indicate how would you interconnect four such adders in order to build a 4-bit Ripple-Carry-Adder for which a symbol is depicted in Figure 3(b).

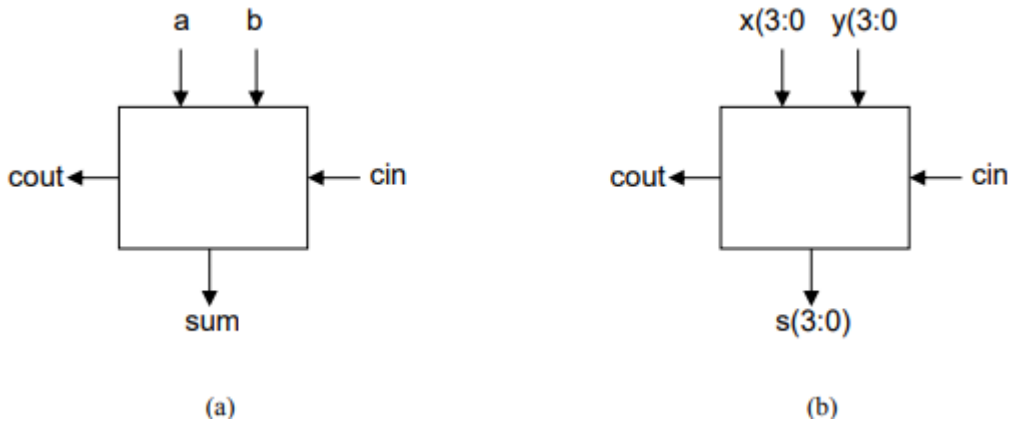

Figure 3: (a) 1-bit Full Adder Symbol, (b) 4-bit Ripple Carry Adder Symbol

Write a Verilog code using the full adder you designed above to implement 4-bit ripple carry adder (RCA).

Use **structural (hierarchical) design principles** to create the 4-bit adder/subtractor circuit in Figure 1. Please note that instead of "CIN" signal (Carry-IN) used in the RCA, use "OP" (operation) signal as depicted in Figure 2. This will make it easier to get the 2's complement of a number when it is necessary. Note that when the "OP" signal changes, the operation performed also changes. Also, please consider that "OP" signal also affects one of the input signals coming to the RCA.

Please note that the Adder/Subtractor has two 4-bit 2-to-1 muxes and one 1-bit 2-to-1 mux.

Because the ALU has an 8-bit output, yet the Adder/Subtractor has a 4-bit output, as well as Cout, you will need to pad it with zeros to make it 8 bits.

Your Adder/Subtractor module will look something like this:

```
module adder_subtractor (input [3:0] A,B, input OP, output [7:0] S);
    mux2to1_4bit m1 (    );
    ripple_carry_adder_4bit rca1 (    );
    and g1 (sel, ~Cout, OP);
    ripple_carry_adder_4bit rca2 (    );
    mux2to1_4bit m2 (    );
    mux2to1_1bit m3 (    );
    assign S = {(3'b000),Cout,Sum}; //concatenating zeros, Cout and the 4-
bit sum/difference to make the 8-bit output
endmodule
```

2.4.2. Design the Datapath

- Add a new Verilog file (e.g., calculator_datapath.v) that includes your top-level module.
- Instantiate the sub-modules (decoder, muxes, flip-flops, ALU) and connect them as specified in the schematic.
- Comment your code thoroughly. For example:

```
// Instantiate the 1-to-2 decoder to determine register write address

decoder my_decoder(

  .wrt_addr(wrt_addr),

  .data_selA(data_selA),

  .data_selB(data_selB)

);
```

2.4.3. Simulation with ModelSim

- Create a testbench file (e.g., `tb_calculator_datapath.v`).
- Provide stimulus for each arithmetic operation (addition, subtraction, multiplication, division).
- Simulate different `wrt_addr`, `op_code`, and `compute_op` combinations.
- Include comments to indicate which operation is being tested.

Compile and simulate in ModelSim. Capture waveforms showing inputs, register values, ALU outputs, and status signals for each operation. Include these in your report with explanations.

2.5. PROJECT REPORT REQUIREMENTS

Your final report should be well-structured and include the following sections:

Title Page

- Project title, your name, course, and date.

Objective Statement

- A brief description of what you aimed to achieve in the project.

Introduction

- Overview of the key concepts (structural Verilog, simulation, and FPGA implementation).

Body

- Verilog Code: Include all your Verilog and test bench codes (properly commented).
- Simulation Results: Provide screenshots of waveform outputs from ModelSim for each submodule along with explanations of what each waveform demonstrates.

Conclusion

- Summarize your findings, challenges faced, and how you resolved them.
- Reflect on the integration process and any potential improvements for future projects.

2.6. REFERENCES

- Digital Design with An Introduction to the Verilog HDL, 5th Edition, M. Morris Mano & Michael D. Ciletti, Pearson